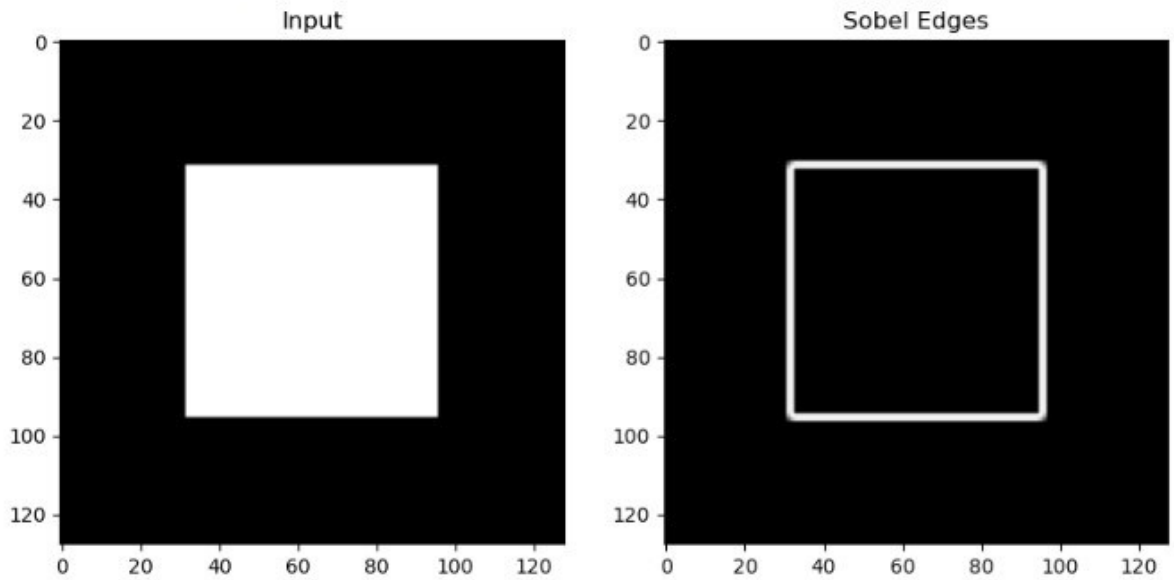
EXPERIMENT 3:

```
import numpy as np from
scipy import ndimage import
matplotlib.pyplot as plt import
cv2 def
sobel_edge_detection(image
):
   sx=np.array([[-1,0,1],[-2,0,2],[-1,0,1]])
sy=np.array([[-1,-2,-1],[0,0,0],[1,2,1]])
   Ix=ndimage.convolve(image,sx,mode='constant')
Iy=ndimage.convolve(image,sy,mode='constant')
magnitude=np.hypot(Ix,Iy)     return magnitude/magnitude.max()
img=np.zeros((128,128)) img[32:96,32:96]=1.0
edges=sobel_edge_detection(img) plt.figure(figsize=(10,5))
plt.subplot(121);plt.imshow(img,cmap='gray');plt.title("Input")
plt.subplot(122);plt.imshow(edges,cmap='gray');plt.title("Sobel
Edges") plt.show
```

```
<function matplotlib.pyplot.show(close=None, block=None)>
```

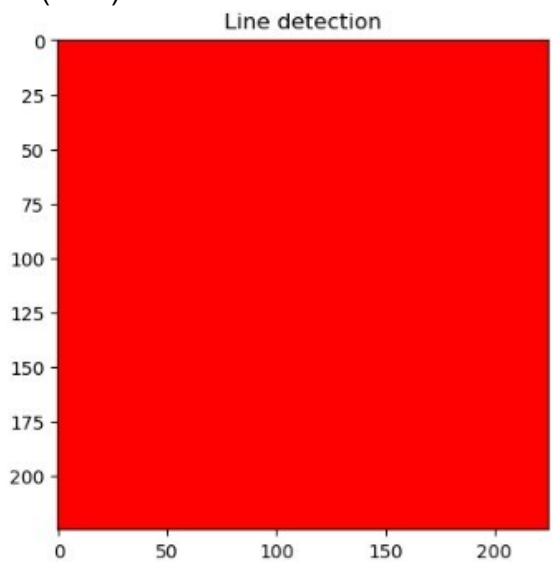



```
def hough_line_detection(binary_edges,theta_step=0.5,threshold_ratio=0.6):
   h,w=binary_edges.shape
diag=int(np.ceil(np.hypot(h,w)))     rhos=np.linspace(-
diag,diag,2*diag+1)     thetas=np.deg2rad(np.arange(-
90,90,theta_step))
accumulator=np.zeros((len(rhos),len(thetas)),dtype=int
)     y,x=np.nonzero(binary_edges>0.1)
   for px,py in zip(x,y):         for t_idx , theta in
enumerate(thetas):
rho=px*np.cos(theta)+py*np.sin(theta)
rho_idx=np.argmin(np.abs(rhos-rho))
accumulator[rho_idx,t_idx]+=1
max_votes=accumulator.max()
```

```
peaks=np.argwhere(accumulator>=threshold_ratio*max_vot
es)     lines=[]     for rho_idx,theta_idx in peaks:

lines.append((rhos[rho_idx],thetas[theta_idx]))
return lines,accumulator

img=cv2.imread("Downloads/lion_line.jpg")
edges=cv2.Canny(img,100,200)
lines,accumulator=hough_line_detection(edges)
print("Detected Lines (rho,theta):",lines)
```

```
Detected Lines (rho,theta): [(np.float64(-105.0), np.float64(-1.53588974175501)), (np.float64(-88.0), np.float64(-1.5707963267948966)), (np.float64(-8
7.0), np.float64(-1.4486232791552935)), (np.float64(-86.0), np.float64(-1.5533430342749532)), (np.float64(-85.0), np.float64(-1.5707963267948966)), (n
p.float64(-84.0), np.float64(-1.53588974175501)), (np.float64(-81.0), np.float64(-1.5533430342749532)), (np.float64(-81.0), np.float64(-1.5358897417550
1)), (np.float64(-80.0), np.float64(-1.5446163880149817)), (np.float64(-80.0), np.float64(-1.5271630954950384)), (np.float64(-78.0), np.float64(-1.5358
8974175501)), (np.float64(-77.0), np.float64(-1.5271630954950384)), (np.float64(-77.0), np.float64(-1.4049900478554354)), (np.float64(-76.0), np.float6
4(-1.5707963267948966)), (np.float64(-76.0), np.float64(-1.5446163880149817)), (np.float64(-76.0), np.float64(-1.5271630954950384)), (np.float64(-75.
0), np.float64(-1.5533430342749532)), (np.float64(-75.0), np.float64(-1.53588974175501)), (np.float64(-75.0), np.float64(-1.4922565104551517)), (np.flo
at64(-75.0), np.float64(-1.4748032179352084)), (np.float64(-74.0), np.float64(-1.562069680534925)), (np.float64(-74.0), np.float64(-1.527163095495038
4)), (np.float64(-74.0), np.float64(-1.509709802975095)), (np.float64(-74.0), np.float64(-1.4835298641951802)), (np.float64(-73.0), np.float64(-1.47480
32179352084)), (np.float64(-73.0), np.float64(-1.457349925415265)), (np.float64(-72.0), np.float64(-1.4660765716752369)), (np.float64(-72.0), np.float6
4(-1.4486232791552935)), (np.float64(-71.0), np.float64(-1.509709802975095)), (np.float64(-70.0), np.float64(-1.509709802975095)), (np.float64(-69.0),
np.float64(-1.53588974175501)), (np.float64(-69.0), np.float64(-1.5009831567151235)), (np.float64(-69.0), np.float64(-1.4922565104551517)), (np.float64
(-69.0), np.float64(-1.4748032179352084)), (np.float64(-69.0), np.float64(-1.4398966328953218)), (np.float64(-68.0), np.float64(-1.4922565104551517)),
(np.float64(-68.0), np.float64(-1.4835298641951802)), (np.float64(-67.0), np.float64(-1.5707963267948966)), (np.float64(-67.0), np.float64(-1.562069680
534925)), (np.float64(-67.0), np.float64(-1.4748032179352084)), (np.float64(-66.0), np.float64(-1.4660765716752369)), (np.float64(-66.0), np.float64(-
1.413716694115407)), (np.float64(-65.0), np.float64(-1.5533430342749532)), (np.float64(-65.0), np.float64(-1.4748032179352084)), (np.float64(-65.0), n
p.float64(-1.457349925415265)), (np.float64(-64.0), np.float64(-1.562069680534925)), (np.float64(-64.0), np.float64(-1.5533430342749532)), (np.float64
(-64.0), np.float64(-1.5446163880149817)), (np.float64(-64.0), np.float64(-1.4660765716752369)), (np.float64(-64.0), np.float64(-1.3962634015954636)),
```

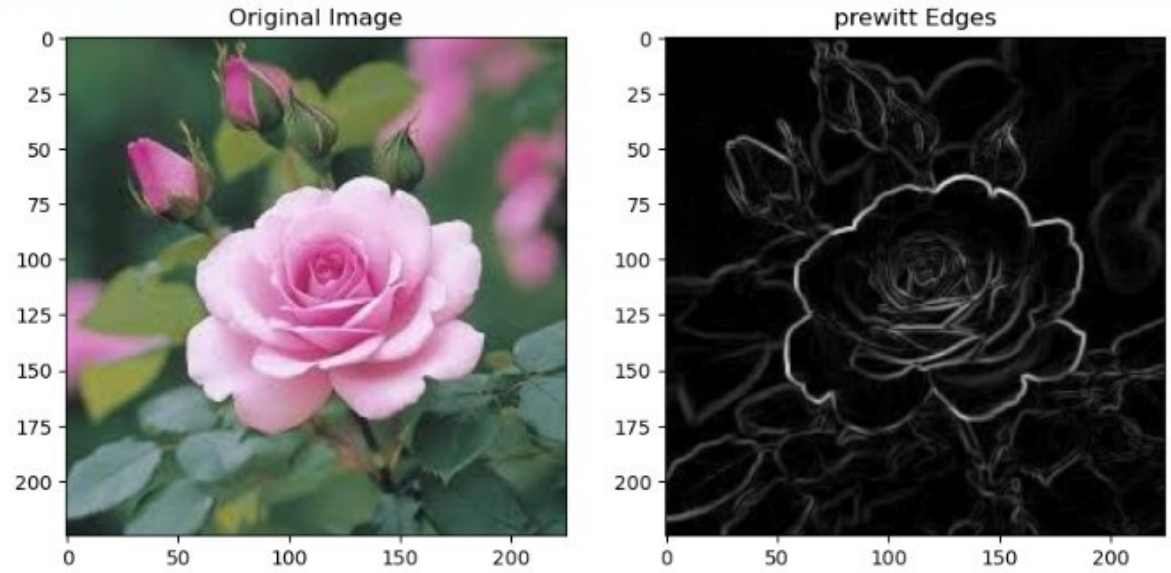
```
img=cv2.imread("Downloads/rose.jpg")  for  rho,theta  in  lines:  a=np.cos(theta)
b=np.sin(theta)  x0=a*rho  y0=b*rho  x1=int(x0+1000*(-b))  y1=int(y0+1000*(a))
x2=int(x0-1000*(-b))  y2=int(y0-1000*(a))  cv2.line(img,(x1,y1),(x2,y2),(0,0,255),2)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB))  plt.title("Line  detection")  plt.show()
print(lines)
```



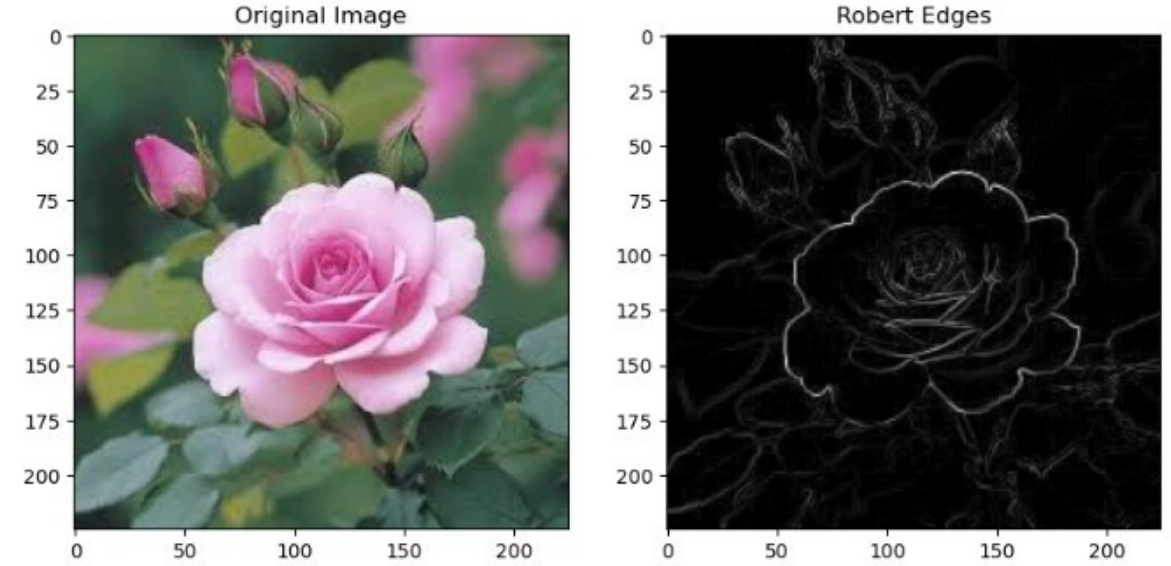

```
[(np.float64(-105.0), np.float64(-1.53588974175501)), (np.float64(-88.0), np.float64(-1.5707963267948966)), (np.float64(-87.0), n
1552935)), (np.float64(-86.0), np.float64(-1.5533430342749532)), (np.float64(-85.0), np.float64(-1.5707963267948966)), (np.float6
1.53588974175501)), (np.float64(-81.0), np.float64(-1.5533430342749532)), (np.float64(-81.0), np.float64(-1.53588974175501)), (np
oat64(-1.5446163880149817)), (np.float64(-80.0), np.float64(-1.5271630954950384)), (np.float64(-78.0), np.float64(-1.535889741755
0), np.float64(-1.5271630954950384)), (np.float64(-77.0), np.float64(-1.4049900478554354)), (np.float64(-76.0), np.float64(-1.570
loat64(-76.0), np.float64(-1.5446163880149817)), (np.float64(-76.0), np.float64(-1.5271630954950384)), (np.float64(-75.0), np.flo
32)), (np.float64(-75.0), np.float64(-1.53588974175501)), (np.float64(-75.0), np.float64(-1.4922565104551517)), (np.float64(-75.0
```

```
img=cv2.imread("Downloads/rose.jpg") gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
px=np.array([[-1,0,1],[-1,0,1],[-1,0,1]],dtype=float) py=np.array([[-1,-1,-
1],[0,0,0],[1,1,1]],dtype=float)
Ix=ndimage.convolve(gray.astype(float),px)
Iy=ndimage.convolve(gray.astype(float),py)
```

```
edges=np.hypot(Ix,Iy)
edges=np.uint8(255*edges/np.max(edges))
plt.figure(figsize=(10,5)) plt.subplot(121)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2R
GB)) plt.title("Original Image") plt.subplot(122)
plt.imshow(edges,cmap='gray') plt.title("prewitt
Edges")
plt.show()
```

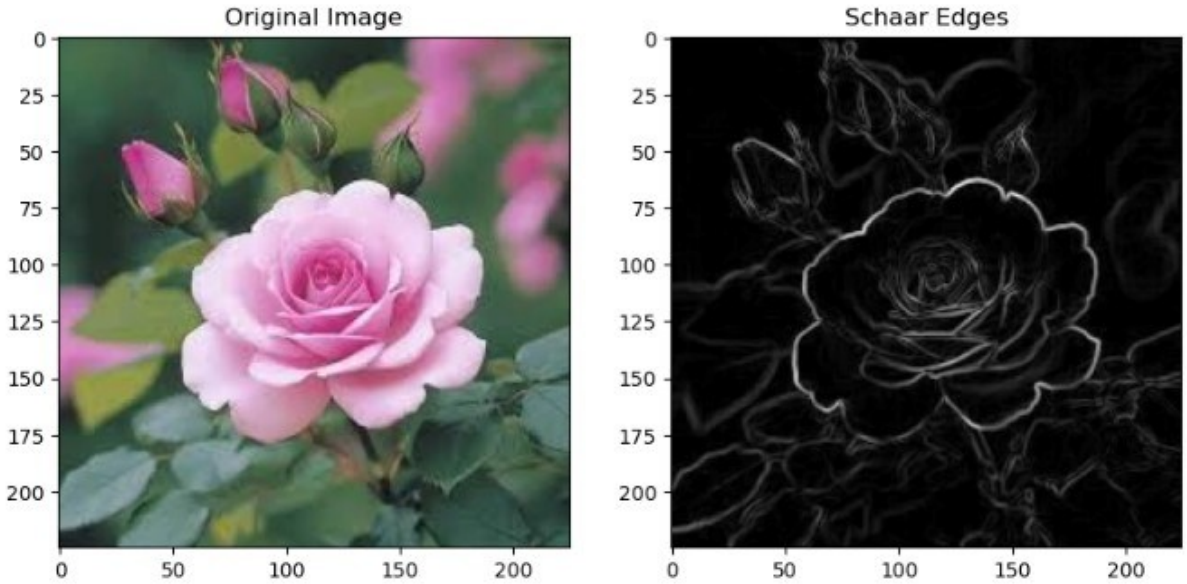



```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY) rx=np.array([[1,0],[0,-1]],dtype=float)
ry=np.array([[0,1],[-1,0]],dtype=float) Ix=ndimage.convolve(gray.astype(float),rx)
Iy=ndimage.convolve(gray.astype(float),ry) edges=np.hypot(Ix,Iy)
edges=np.uint8(255*edges/np.max(edges)) plt.figure(figsize=(10,5)) plt.subplot(121)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)) plt.title("Original Image")
plt.subplot(122) plt.imshow(edges,cmap='gray') plt.title("Robert Edges") plt.show()
```



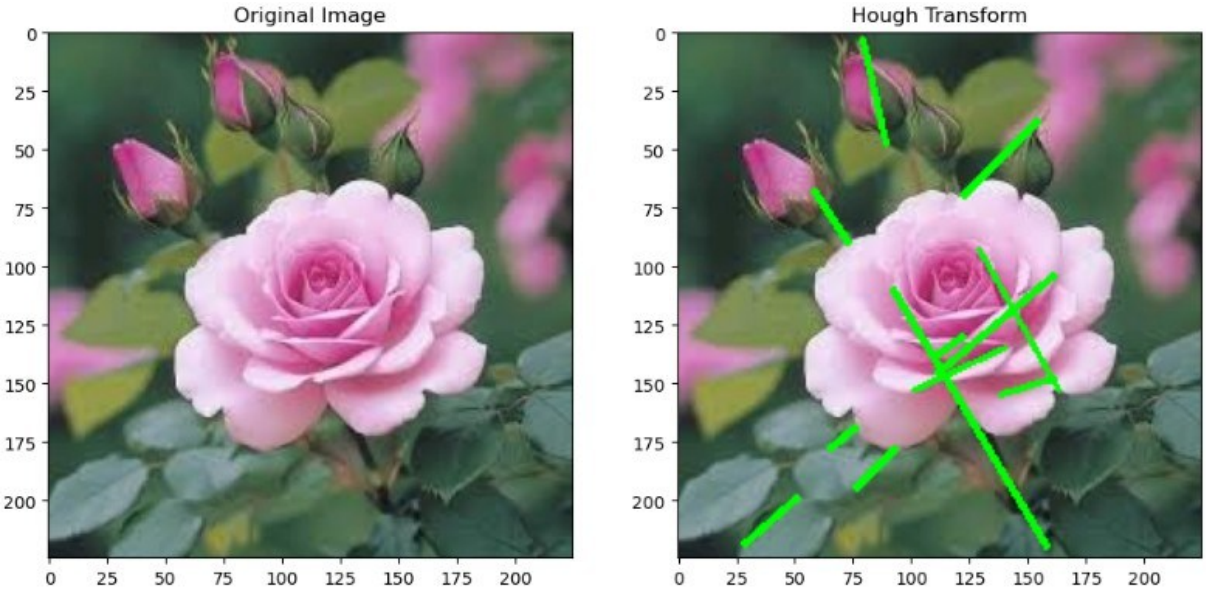

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
```

```
Ix=cv2.Scharr(gray,cv2.CV_64F,1,0)
Iy=cv2.Scharr(gray,cv2.CV_64F,0,1)
edges=cv2.magnitude(Ix,Iy)
edges=np.uint8(255*edges/np.max(edges))
plt.figure(figsize=(10,5)) plt.subplot(121)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2R
GB)) plt.title("Original Image") plt.subplot(122)
plt.imshow(edges,cmap='gray') plt.title("Schaar
Edges") plt.show()
```



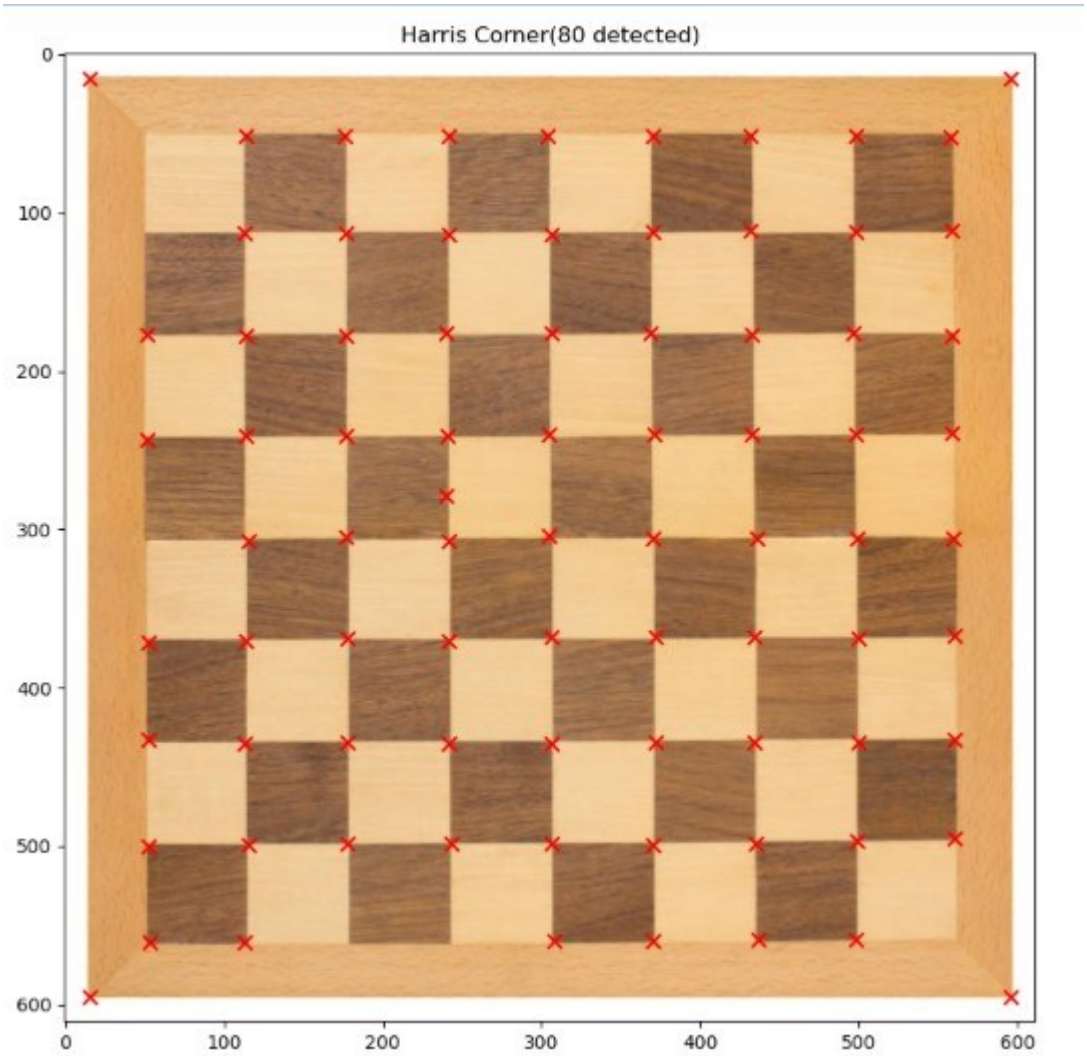

```
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
gray=cv2.GaussianBlur(gray,(5,5),0)

edges=cv2.Canny(gray,80,150)

lines=cv2.HoughLinesP(edges,1,np.pi/180,threshold=50,minLineLength=10,maxLineGap=10
)

line_img=img.copy()

if lines is not None:
    for x1,y1,x2,y2 in lines[:,0]:
        cv2.line(line_img,(x1,y1),(x2,y2),(0,255,0),2)

plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2R
GB)) plt.title("Original Image")

plt.subplot(1,2,2)
plt.imshow(cv2.cvtColor(line_img,cv2.COLOR_BGR2RGB))
plt.title("Hough Transform")
```

plt.show()



img=cv2.imread("Downloads/chess.jpg")
gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY).astype(np.float32)/255.0
Ix=ndimage.sobel(gray,axis=1)
Iy=ndimage.sobel(gray,axis=0)
sigma=1.5
Ix2=ndimage.gaussian_filter(Ix**2,sigma)
Iy2=ndimage.gaussian_filter(Iy**2,sigma)
Ixy=ndimage.gaussian_filter(Ix*Iy,sigma)

k=0.05 det=Ix2*Iy2-
Ixy**2 trace=Ix2+Iy2
R=det-k*trace**2
R_thresh=np.where(R>0.15*R.max(),R,0)
R_max=ndimage.maximum_filter(R_thresh,size=1
0) local_max=(R_thresh==R_max)&(R_thresh>0)
corners=np.argwhere(local_max)
plt.figure(figsize=(8,8))
plt.imshow(cv2.cvtColor(img,cv2.COLOR_BGR2R
GB))
plt.scatter(corners[:,1],corners[:,0],c='red',s=60,marker='x',linewidths
=1.5) plt.title(f"Harris Corner({len(corners)} detected)")
plt.tight_layout() plt.show()


Harris Corner(80 detected)
0
100
200
300
400
500
600
0
100
200
300
400
500
600